

Phase7 ConfusionBuffer Anki

Phase 7 — Confusion Buffer & Anki Pack (Computer Vision Expert: classical / geometric / 3D / edge)

Companion to the Phase-7 daily schedule (Weeks 64-69). Rounds out CV breadth (video already covered in Phase 2V) + real deployment skill. (Expanded edition.)

How to use: as prior packs. **Triangulation targets:** the *pinhole model* + *epipolar constraint*, *essential vs fundamental matrix*, *SIFT scale/rotation invariance*, *two-stage vs one-stage detectors*, and *what TensorRT does* + *INT8 calibration*.

Ranked hard-topics map

1. **Multi-view geometry** — pinhole, epipolar, F vs E, triangulation, PnP.
2. **Features + robust estimation** — SIFT/ORB, RANSAC, homography.
3. **Detection/segmentation** — two/one-stage, anchors, NMS, mAP, DETR/Mask R-CNN.
4. **Deployment/edge** — ONNX→TensorRT, quantization/calibration, latency profiling.
5. **3D/neural rendering** — NeRF vs 3DGS, point-cloud registration.

Anki deck (Q → A)

Deck A • Classical & geometric CV

- **Q:** Pinhole camera model? → **A:** 3D→2D projection: $x = K[R|t]X$ (homogeneous); K = intrinsics (focal length, principal point), $[R|t]$ = extrinsics (pose).
- **Q:** Intrinsics vs extrinsics? → **A:** Intrinsics (K) = camera-internal (focal, principal point, skew); extrinsics ($[R|t]$) = camera pose in the world.
- **Q:** Lens distortion — types + fix? → **A:** Radial (barrel/pincushion) + tangential; calibrate with a checkerboard to estimate distortion coefficients and undistort.
- **Q:** Epipolar constraint + fundamental matrix? → **A:** For corresponding points across two views, $x^T F x' = 0$; F reduces correspondence search to the epipolar line.
- **Q:** Fundamental vs essential matrix? → **A:** F works on pixel coordinates (uncalibrated); $E = K^T F K$ works on normalized coordinates (calibrated) and decomposes into R, t .
- **Q:** How does SIFT achieve scale/rotation invariance? → **A:** Keypoints across a scale-space (Difference-of-Gaussians), a dominant orientation, and a local gradient-orientation histogram descriptor → invariant to scale/rotation, robust to illumination.
- **Q:** SIFT vs ORB — trade-off? → **A:** SIFT = robust, higher quality, slower (and historically patented); ORB = fast, binary descriptor (Hamming matching), great for real-time/SLAM.
- **Q:** When is a homography valid? → **A:** A projective 3×3 map between planes — valid for planar scenes or pure-rotation camera motion.
- **Q:** What does RANSAC do? → **A:** Robustly fits a model (F /homography) by repeatedly sampling minimal point sets, counting inliers, keeping the best hypothesis → tolerates outlier matches.
- **Q:** Triangulation? → **A:** Given corresponding points + two known camera matrices, solve for the 3D point (intersection of back-projected rays, least-squares).
- **Q:** What is PnP (Perspective-n-Point)? → **A:** Estimate a camera's pose (R, t) from n known 3D points and their 2D projections — used to register a new camera/frame.

Deck B • Structure-from-Motion / SLAM

- **Q:** The SfM pipeline? → **A:** Feature detect+match (+RANSAC) → two-view geometry → triangulate 3D points → incrementally register cameras (PnP) → bundle adjustment.
- **Q:** What is bundle adjustment? → **A:** Jointly optimize camera poses + 3D points to minimize total reprojection error (large sparse nonlinear least squares).
- **Q:** SLAM vs SfM? → **A:** SLAM = real-time, sequential, online (robotics/AR); SfM = offline batch reconstruction.
- **Q:** What are keyframes + why? → **A:** A subset of representative frames used for mapping/optimization → bounds cost and drift vs using every frame.
- **Q:** What is loop closure + why? → **A:** Recognizing a revisited place (place recognition / bag-of-words) and adding a constraint → corrects accumulated drift.
- **Q:** Visual-inertial odometry — why add an IMU? → **A:** Fuses camera with inertial measurements → robust to motion blur/low texture and gives metric scale.
- **Q:** Sparse vs dense reconstruction? → **A:** Sparse = a point cloud from features (SfM/SLAM); dense = per-pixel depth/mesh (MVS) — more complete, more compute.

Deck C • Detection & segmentation

- **Q:** Two-stage vs one-stage detectors? → **A:** Two-stage (Faster R-CNN): region proposals then classify/refine (accurate); one-stage (YOLO/RetinaNet): dense prediction in one pass (fast).
- **Q:** Anchor-based vs anchor-free? → **A:** Anchors = predefined boxes to regress from; anchor-free (FCOS/CenterNet) predicts centers/points directly (simpler, fewer hyperparameters).
- **Q:** What is NMS? → **A:** Non-Max Suppression — keep the highest-score box, remove others overlapping it above an IoU threshold → dedupe detections.
- **Q:** IoU + mAP? → **A:** IoU = intersection/union of boxes; mAP = mean Average Precision (area under precision-recall per class, over IoU thresholds), averaged across classes.
- **Q:** Why focal loss in dense detection? → **A:** Down-weights easy background examples (the vast majority) so training focuses on hard/rare foreground → fixes extreme foreground/background imbalance.
- **Q:** What does a Feature Pyramid Network (FPN) add? → **A:** Multi-scale features (combine high-res/low-semantic + low-res/high-semantic) →

detect objects across scales.

- **Q:** Mask R-CNN — what over Faster R-CNN? → **A:** Adds a per-RoI mask branch (+ RoIAlign) → instance segmentation alongside detection.
- **Q:** DETR — the shift? → **A:** Transformer detection as set prediction (learned object queries + bipartite Hungarian matching) → no anchors/NMS.
- **Q:** Semantic vs instance vs panoptic segmentation? → **A:** Semantic = per-pixel class (no instances); instance = separate objects; panoptic = both (stuff + things).

Deck D · Deployment & edge

- **Q:** What does TensorRT do? → **A:** Optimizes a trained net for NVIDIA GPUs — layer/tensor fusion, precision calibration (FP16/INT8), kernel auto-tuning → lower latency/higher throughput.
- **Q:** ONNX role? → **A:** A framework-agnostic interchange format to export a model and run it in an optimized runtime (TensorRT/ONNXRuntime).
- **Q:** INT8 calibration — how + why? → **A:** Run representative data to collect activation ranges → choose per-tensor quantization scales that minimize accuracy loss (post-training quantization).
- **Q:** FP16 vs INT8 vs INT4 — trade-off? → **A:** Lower precision = smaller/faster with rising accuracy risk; FP16/BF16 usually near-lossless, INT8 small loss (needs calibration), INT4 aggressive (needs care/QAT).
- **Q:** Why port PyTorch→C++/TensorRT for real-time? → **A:** Remove Python overhead, fuse ops, use lower precision → meet strict latency/throughput on-device.
- **Q:** What is memory-bandwidth bound (edge)? → **A:** When data movement, not FLOPs, limits speed → fusion, quantization, and smaller models help more than raw compute.
- **Q:** How do you profile latency properly? → **A:** Warm up, measure p50/p95/p99 (not just mean), fixed batch/input shapes, and separate host↔device transfer from compute.
- **Q:** Distillation/pruning for edge? → **A:** Distill a small student from a big teacher; prune redundant weights/channels → smaller, faster models with modest accuracy cost.
- **Q:** What does Triton Inference Server provide? → **A:** Serving multiple models with dynamic batching, concurrency, and multiple backends behind one endpoint.

Deck E · 3D vision & neural rendering

- **Q:** What is NeRF? → **A:** An MLP mapping (3D position, view direction) → (color, density); volume-render rays to synthesize novel views; trained per-scene from posed images (slow).
- **Q:** The volume-rendering idea (one line)? → **A:** Integrate color weighted by transmittance and density along each ray → a differentiable rendering of the scene.
- **Q:** Why positional encoding in NeRF? → **A:** Raw coordinates make the MLP low-frequency/blurry; a Fourier positional encoding lets it represent high-frequency detail.
- **Q:** 3D Gaussian Splatting vs NeRF? → **A:** Represent a scene as many 3D Gaussians and rasterize (not ray-march) → real-time, high-quality novel views, much faster than classic NeRF.
- **Q:** Meshes vs point clouds vs voxels vs SDF? → **A:** Mesh = surface (vertices+faces, compact); point cloud = unstructured points (from sensors/SfM); voxels = 3D grid (dense, memory-heavy); SDF = signed distance field (smooth implicit surface).
- **Q:** What does ICP do (point clouds)? → **A:** Iterative Closest Point — repeatedly match nearest points and solve the rigid transform to align two clouds (point-to-point or point-to-plane).

Common misconceptions & traps

- **"Deep learning replaced geometry."** Multi-view geometry still underpins SfM/SLAM/calibration/3D — DL complements it.
- **"Higher mAP = better for deployment."** Latency, model size, and power often decide the shipped model.
- **"Quantization is lossless."** It trades accuracy; calibrate + validate (and consider QAT for INT4).
- **"NeRF is real-time."** Classic NeRF is slow to train/render; 3D Gaussian Splatting is the real-time option.
- **"Mean latency is the metric."** Tail latency (p95/p99) governs user experience and SLAs.
- **"F and E matrices are interchangeable."** F is uncalibrated (pixels); E needs K (calibrated) and yields R,t.

Glossary starter

pinhole model / intrinsics-extrinsics · radial/tangential distortion / calibration · epipolar constraint · fundamental vs essential matrix · SIFT (DoG, orientation) / ORB · homography · RANSAC · triangulation · PnP · SfM / bundle adjustment / reprojection error · SLAM / keyframes / loop closure / bag-of-words · visual-inertial · sparse vs dense (MVS) · two-stage vs one-stage · anchor-based/-free · NMS · IoU / mAP · focal loss · FPN · Mask R-CNN / RoIAlign · DETR (set prediction, Hungarian) · semantic/instance/panoptic · ONNX · TensorRT / fusion / INT8 calibration / QAT · FP16/INT8/INT4 · memory-bandwidth bound · p95/p99 profiling · distillation/pruning · Triton · NeRF / volume rendering / positional encoding · 3D Gaussian Splatting · mesh/point cloud/voxel/SDF · ICP.

Drills

Whiteboard: the pinhole model + epipolar constraint + F-vs-E; SIFT invariance; the SfM pipeline + bundle adjustment; two-stage vs one-stage + NMS + mAP; what TensorRT does + INT8 calibration. **Blank-file:** corner/edge detectors + a RANSAC homography; run COLMAP SfM on an image set; NMS + IoU from scratch; export a model to ONNX + build a TensorRT engine and benchmark p50/p95/p99; ICP on two point clouds.

Leaving bar (cold, no notes)

Pinhole + epipolar + F-vs-E + triangulation/PnP; SIFT scale/rotation invariance + RANSAC; the SfM pipeline + bundle adjustment + loop closure; two-stage vs one-stage detectors + NMS + mAP + FPN + DETR; what TensorRT does + INT8 calibration + tail-latency profiling; port a PyTorch CV model to a real-time C++/TensorRT pipeline; NeRF vs 3DGS + ICP.